

Your Library — Revision 2.7.0

Isometric Library Room — Integration Build Prompt

Goal	Integrate the standalone isometric Room into the app's Room tab
Source files	scene.jsx, furniture.jsx, panels.jsx, tweaks-panel.jsx, data.jsx, Library_Room.html
Tech match	Pure React 18 + browser Babel + window-globals (same as index.html)
3D engine	NONE — CSS/SVG isometric projection. No Three.js / WebGL / Spline.
New persistence	One new Supabase table: rooms

Context for Claude Code

I have a standalone isometric "library room" feature built as a separate React app (files: scene.jsx, furniture.jsx, panels.jsx, tweaks-panel.jsx, data.jsx, and a host page Library_Room.html). I want it integrated into my main app (index.html) as the content of the existing "Room" tab in the user profile — replacing the current "Coming Soon" stub from v2.5.1.

Critical facts about both codebases:

- The room is NOT 3D/WebGL. It is a CSS + SVG isometric render. There are no external 3D libraries to install.
- The room uses the EXACT same stack as index.html: React 18 UMD, @babel/standalone (in-browser Babel), <script type="text/babel">, and the Object.assign(window, {...}) global-registration pattern.
- Room components self-register on window: Scene, Furniture, BookSpine, tileTolso, shade, Inventory, ShelfDetail, BookDetail, Toolbar, useTweaks (from tweaks-panel).
- data.jsx is mock data only. Its header comments confirm field names were chosen to match the real backend. It must be replaced with live Supabase data, not pasted in.

Step 1 — Inline the component files into index.html

Paste the contents of these four files into the main <script type='text/babel'> block in index.html, BEFORE the main App component is defined (so the window globals exist when the app renders). Paste in this order to respect dependencies:

- tweaks-panel.jsx (defines useTweaks hook + tweak UI)
- furniture.jsx (defines Furniture, BookSpine, tileTolso, shade)
- panels.jsx (defines Inventory, ShelfDetail, BookDetail, Toolbar)
- scene.jsx (defines Scene — depends on Furniture)

Do NOT paste data.jsx. Do NOT include the React/Babel CDN script tags from Library_Room.html — index.html already loads them.

Important — JSX no-conflict check: these files use bare React.useState / React.useRef etc., which is fine. But confirm no top-level const/function name in the room files collides with an existing name in index.html (e.g. a second 'App', 'Toolbar', or 'BookDetail'). If a collision exists, rename the room's

version with a Room prefix (RoomToolbar, RoomBookDetail, etc.) and update its references + its window registration.

Step 2 — Scope the room's CSS (highest-risk step)

Read the CSS Carefully

The room's stylesheet (in the <style> block of Library_Room.html) uses generic class names that almost certainly collide with the main app, including: .active, .hidden, .dim, .ghost, .toolbar, .book-detail, .book-spine, .app-avatar, .app-nav. Pasting them raw WILL break styling in one or both UIs.

Required approach:

- Wrap the entire mounted room in a single container: ...
- Prefix every selector from the room's stylesheet with .libroom-root (e.g. .toolbar becomes .libroom-root .toolbar). Scope ALL of it, not just the risky ones — it's safer and mechanical.
- The room sets document.body.className = 'theme-cottage' etc. for theming (see App in Library_Room.html). Do NOT let it write to document.body — that will nuke the main app's body classes. Instead, apply the theme class to the .libroom-root wrapper and refactor the body-class effect to target the wrapper element via a ref.
- Same for the edit-mode body class ('edit-on') and fullscreen-mode: move them onto .libroom-root, and update scene.jsx which checks document.body.classList.contains('edit-on') — point it at the wrapper instead.

Step 3 — Create the rooms table in Supabase

The room layout is the only genuinely new data. Currently the standalone app persists to localStorage ('library-room-state'). Replace that with a Supabase table:

```
rooms
  id          uuid PK default gen_random_uuid()
  user_id     uuid FK -> auth.users  (UNIQUE — one room per user)
  size        jsonb  -- { w, d }
  tile_size   integer
  wall_color  text
  floor_style text
  window_wall text
  theme       text    -- 'cottage' | 'academia' | 'modern'
  items       jsonb    -- the items[] array (furniture placements)
  audio       jsonb    -- { enabled, track, volume }
  created_at  timestampz default now()
  updated_at  timestampz default now()
```

RLS policies:

- SELECT: a room is readable if the owner's profile visibility allows it (reuse the SAME visibility/privacy logic already built in v2.5.1 for shelves — public readable by all, friends readable by accepted followers, private/hidden owner-only). Mirror the existing SECURITY DEFINER helper pattern already used on books + shelves.
- INSERT / UPDATE / DELETE: only where user_id = auth.uid().

On first visit to a user's own Room tab with no row present, create one from DEFAULT_ROOM (port the constant out of data.jsx into the app as ROOM_DEFAULT). Debounce saves (the standalone uses a 250ms debounce — keep that, but write to Supabase instead of localStorage).

Step 4 — Replace mock data with live library data

data.jsx exposes four globals the room reads: MOCK_BOOKS, MOCK_SHELVES, MOCK_ACCOUNT, DEFAULT_ROOM, CATALOG. Wire them to real app state:

- **Books:** feed the room the same books state the app already loads from Supabase. Map fields to the room's expected shape: id, title, author, spine (color), height, width, status, rating, pagesRead, pages, notes, genre.
- **Spine color / height / width:** the room needs a spine color and dimensions per book for rendering. If the app doesn't store these, derive them deterministically: hash the ISBN/title to a hue for spine color, map page count to height/width within sane bounds. Keep it a pure function so the same book always looks the same.
- **Shelves:** feed real shelves; map each to { id, name, sortBy, bookIds, styleId }. styleId can default based on the shelf's existing wood_type (e.g. wood_type 'oak' -> 'shelf-tall-oak').
- **Account:** replace MOCK_ACCOUNT with the real viewed-profile data (id, displayName, privacy/visibility, readingStreak if tracked — if not, omit the streak UI or pass 0).
- **CATALOG:** keep as a shipped static constant (furniture catalog is app content, not user data). The 'My Shelves' catalog category should populate from the user's real shelves so they can drop them into the room.

Status value mapping (do not skip):

The room uses status values 'read' / 'reading' / 'tbr'. The main app uses 'read' / 'currently-reading' / 'to-read'. Add a small bidirectional mapping layer at the data boundary so both stay consistent:

```
const toRoomStatus = { 'read':'read', 'currently-reading':'reading', 'to-read':'tbr' };
const fromRoomStatus = { 'read':'read', 'reading':'currently-reading', 'tbr':'to-read' };
```

Step 5 — Mount the room in the Room tab

- Locate the profile Room tab stub (the v2.5.1 "Coming Soon" placeholder) and replace its body with the room, wrapped in .
- Port the room's top-level App component (from Library_Room.html) in as a component named LibraryRoom. Remove its ReactDOM.createRoot(...).render(...) call — it must render as a child component, not mount its own root.
- LibraryRoom takes props instead of reading mock globals: <LibraryRoom books={...} shelves={...} account={...} room={roomRow} setRoom={...} editable={isOwnProfile} />.
- **Edit vs read-only:** the room's Edit mode (drag furniture, open catalog) must ONLY be available when the viewer is the profile owner. When viewing someone else's room, force editMode=false and hide the Edit/ Catalog toolbar buttons. The visitor sees a static, explorable room (can still click books/shelves to view detail, can pan/zoom, cannot move anything).
- Lazy-mount: only render LibraryRoom when the Room tab is actually active, so the iso scene math + furniture sprites don't run while the user is on Books/Shelves.

What NOT to change

- Do not modify the existing Books or Shelves tabs, search pipeline, scan flow, or import system.
- Do not let the room write to document.body classes, localStorage, or its own ReactDOM root.
- Do not install any npm packages or add CDN scripts — there are no external dependencies.

- Do not change the existing books/shelves Supabase tables. The room reads them; it does not own them. Only the new rooms table is added.

Deliverables when done

- Four room component files inlined into index.html's Babel block, name-collisions resolved.
- Entire room CSS scoped under .libroom-root; body-class side effects refactored onto the wrapper.
- New Supabase rooms table + RLS reusing v2.5.1 visibility logic.
- Live books/shelves/account wired in; status mapping layer; deterministic spine color/size fallback.
- LibraryRoom mounted in the Room tab; edit mode owner-only; visitors get read-only; lazy-mounted on tab activation.

Work incrementally and keep the app bootable at each step: (1) inline + scope CSS with mock data still hardcoded and confirm the room renders inside the tab; (2) swap mock data for live Supabase data; (3) add the rooms table + persistence; (4) gate edit mode by ownership. If anything is ambiguous, match existing app patterns and keep the app rendering — do not refactor unrelated code.